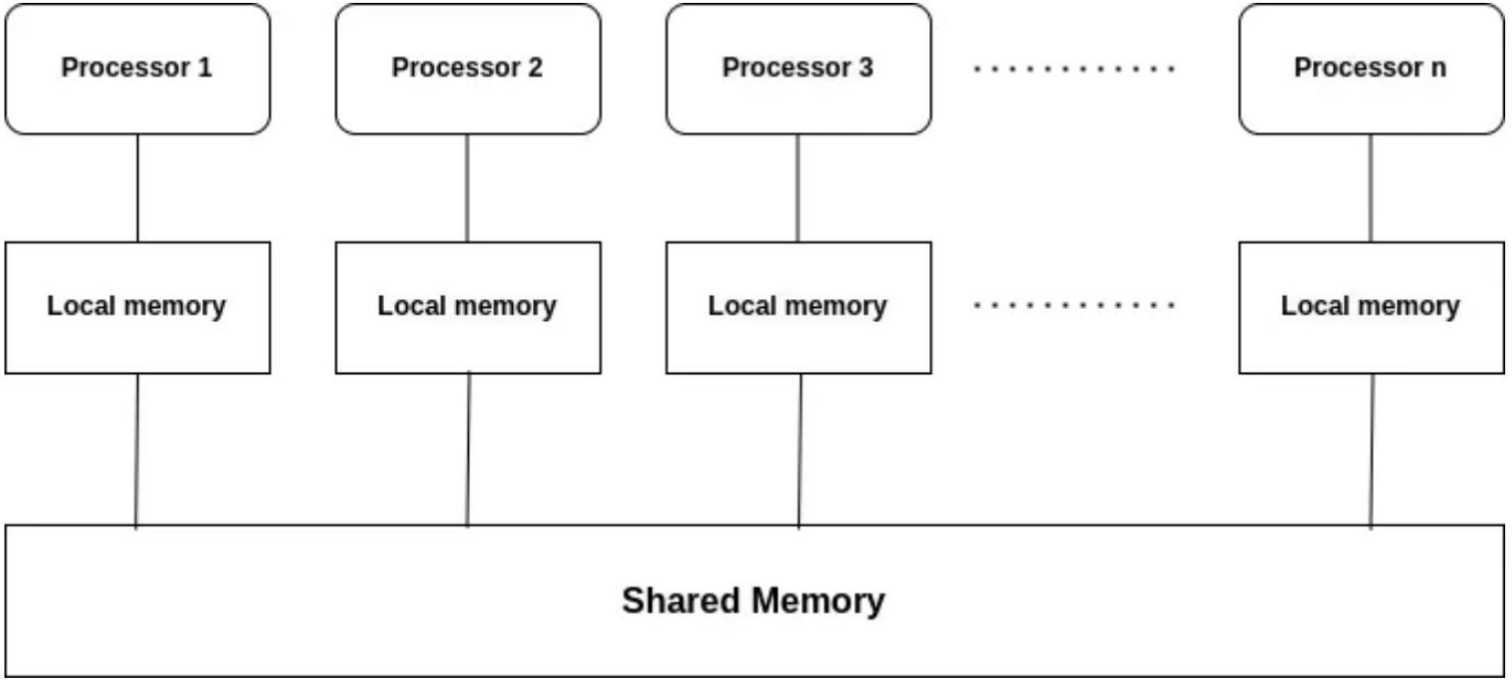
Parallel Computer Memory Architectures

- Shared Memory Architecture
- Distributed Memory Architecture
- Hybrid Architecture

Shared Memory Architecture

- Multiple processors can operate independently but share the same memory resources.
- All the processors can perform read and write operation.
- When one processor updates a value, the others can see the change.
- Similarly to eliminate a value it needs to send data between processors.



Shared Memory Architecture

Advantages:

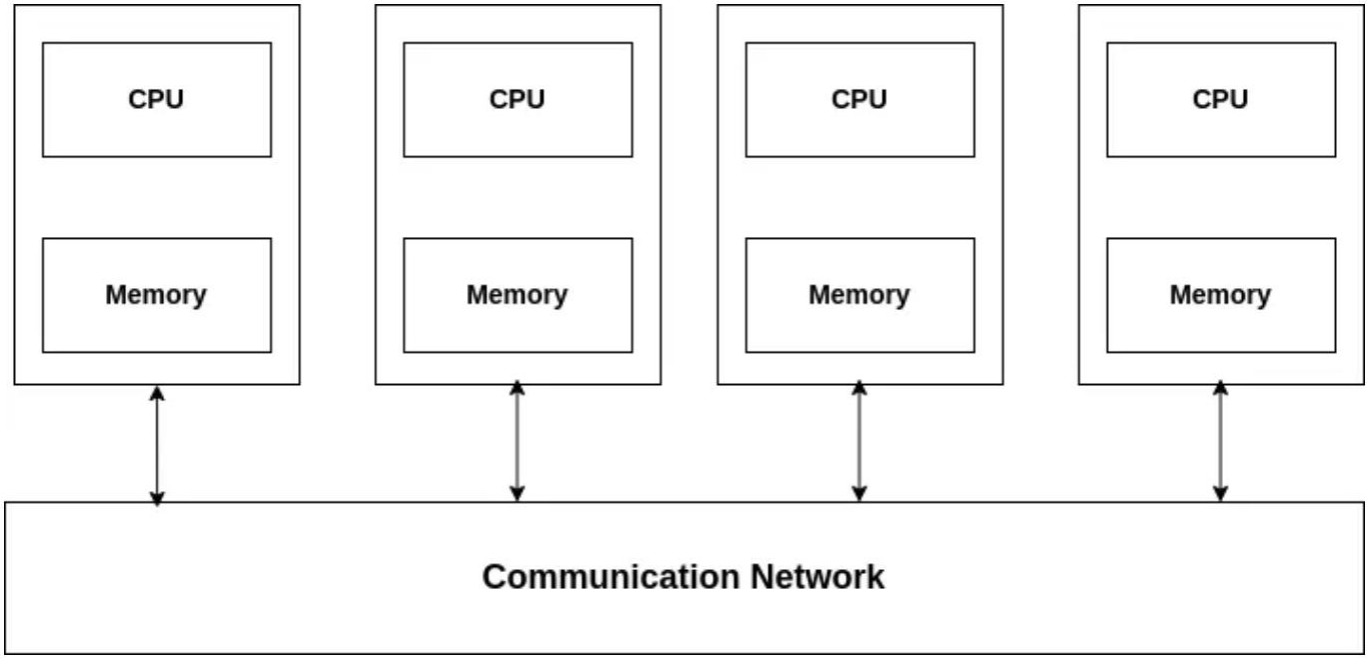
- Fast communication
- Easier to program

Disadvantages:

- Memory bottleneck
- Limited scalability

Distributed Memory Architecture

- Each processor has its own local memory.
- Processors cannot directly access each other's memory.
- If a processor needs data from another, it must send a message through a network.



Distributed Memory Architecture

Advantages:

- ❑ Highly scalable.
- ❑ No memory bottleneck.

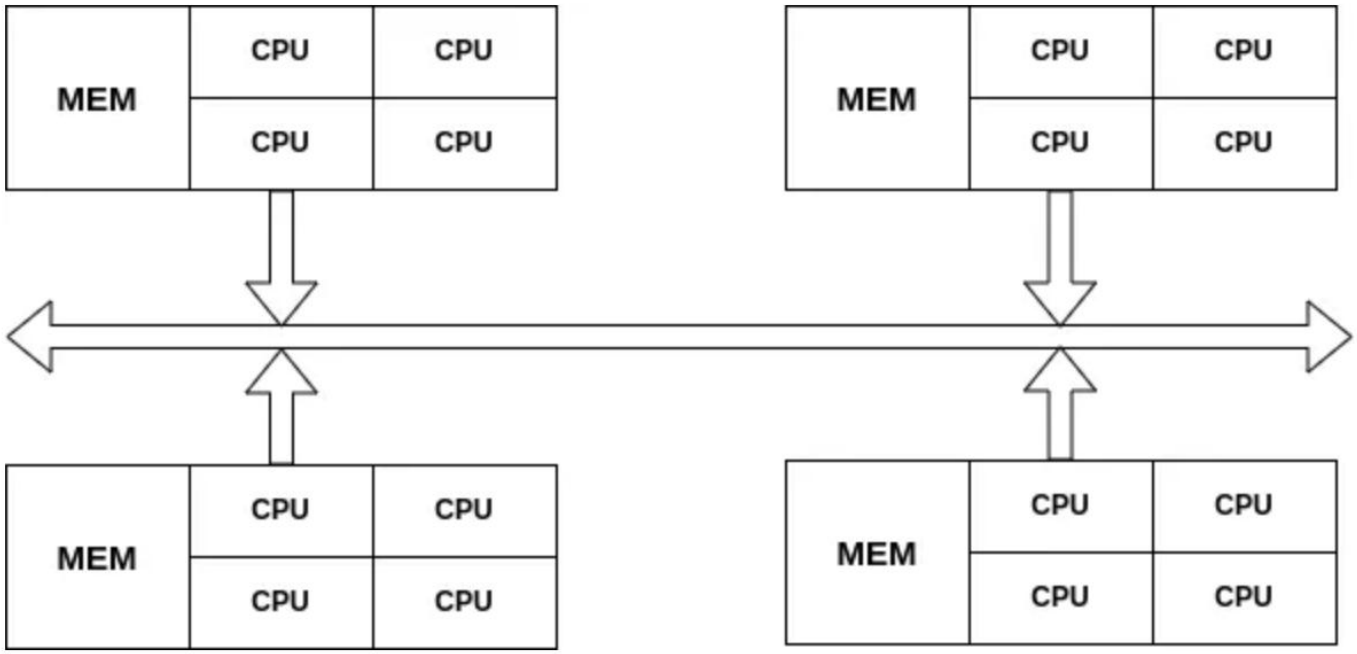
Disadvantages:

- ❑ Difficult to program.
- ❑ Higher communication cost.

Hybrid Architecture

Shared + Distrubuted

- Processors are grouped into small shared-memory clusters.
- These clusters are then connected in a distributed memory network.



Hybrid Architecture

Advantages:

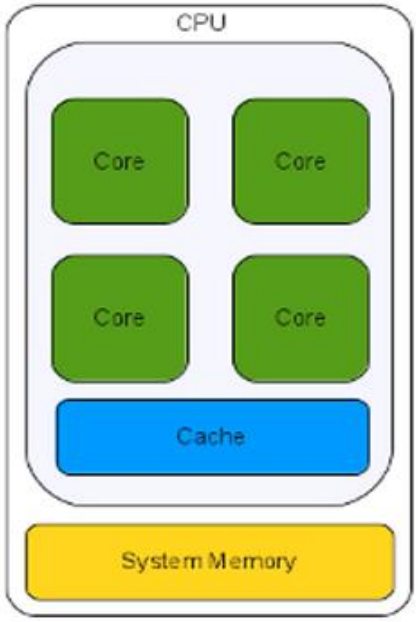
- ❑ Balances speed and scalability
- ❑ Efficient communication

Multicore and Manycore processors

Multicore System

- A **Multicore Processor** has more than one core.

  Example: Dual-core, Quad-core, Octa-core.
- Each core can run instructions simultaneously.
- Instructions types: Calculation, Data transfer & Branch.
- Heat generation is reduced compared to a single fast core.
- It support multithreading, parallel computing



Multicore and Manycore processors

Advantages

- Higher cache coherency due to cores integrated on a single chip.
- More energy efficient with better performance at lower power.
- Less communication traffic between cores.
- Faster processing and improved multitasking.

Disadvantages

- Dual-core processors do not provide exactly double speed.
- Usually give only about **60–80%** performance improvement.
- Some OS still rely mainly on a single core.
- Software OS designed for multicore may run slower on single-core systems.

Multicore and Manycore processors

Application areas

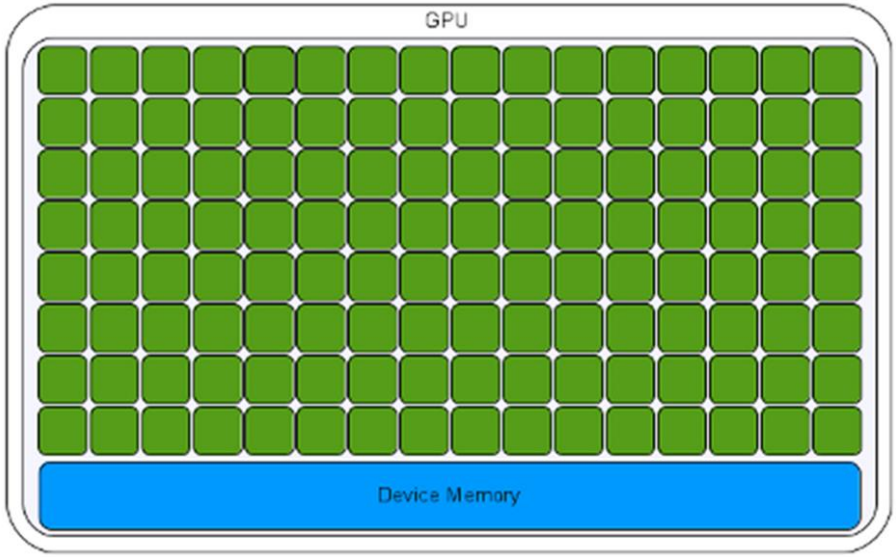
- General-purpose computing
- Embedded systems
- Networking
- Digital Signal Processing (DSP)
- Graphics Processing Units (GPU)

N:B: *Efficient parallel software is needed for better performance.*

Multicore and Manycore processors

Manycore System

- **High Core Count**: It can have hundreds of cores on a single chip, enabling massive parallelism for specialized tasks.
- **Distributed Resources**: Many-core designs often feature distributed caches and memory controllers, allowing each core to access memory independently.
- **Targeted Applications**: Scientific computing, machine learning, and data analytics, where high parallelism can be fully utilized.



Multicore and Manycore processors

Advantages

- **Extreme Parallelism**: It improves performance for highly parallel applications using multiple cores.
- **Scalability**: They scale performance efficiently by adding more cores with less increase in power consumption.
- **Specialized Processing**: Many-core architectures use specialized cores for tasks like graphics and floating-point processing.

Multicore and Manycore processors

Disadvantages

- **Software Optimization:** Efficiently using many cores requires advanced parallel programming.
- **Heat and Power Management:** More cores increase heat and power challenges.
- **Inter-Core Communication:** Fast communication between many cores can become a bottleneck.

Multicore and Manycore processors

Applications

- **High-Performance Computing (HPC):** Used in supercomputers for simulations and scientific research.
- **Artificial Intelligence and Machine Learning:** Ideal for training neural networks and processing large datasets.
- **Embedded Systems:** Used in telecom and automotive systems requiring parallel processing.

GPU and accelerator-based systems

GPU Architecture

- Designed for massive parallel processing & high-speed computation.
- It can handle thousands of tasks simultaneously.
- Used in AI, data analytics, virtualization, and high-performance computing.
- It enable faster data processing, real-time decision-making, and improved system performance.
- Modern platforms like Scale Computing use GPU acceleration for efficient AI-driven workloads.

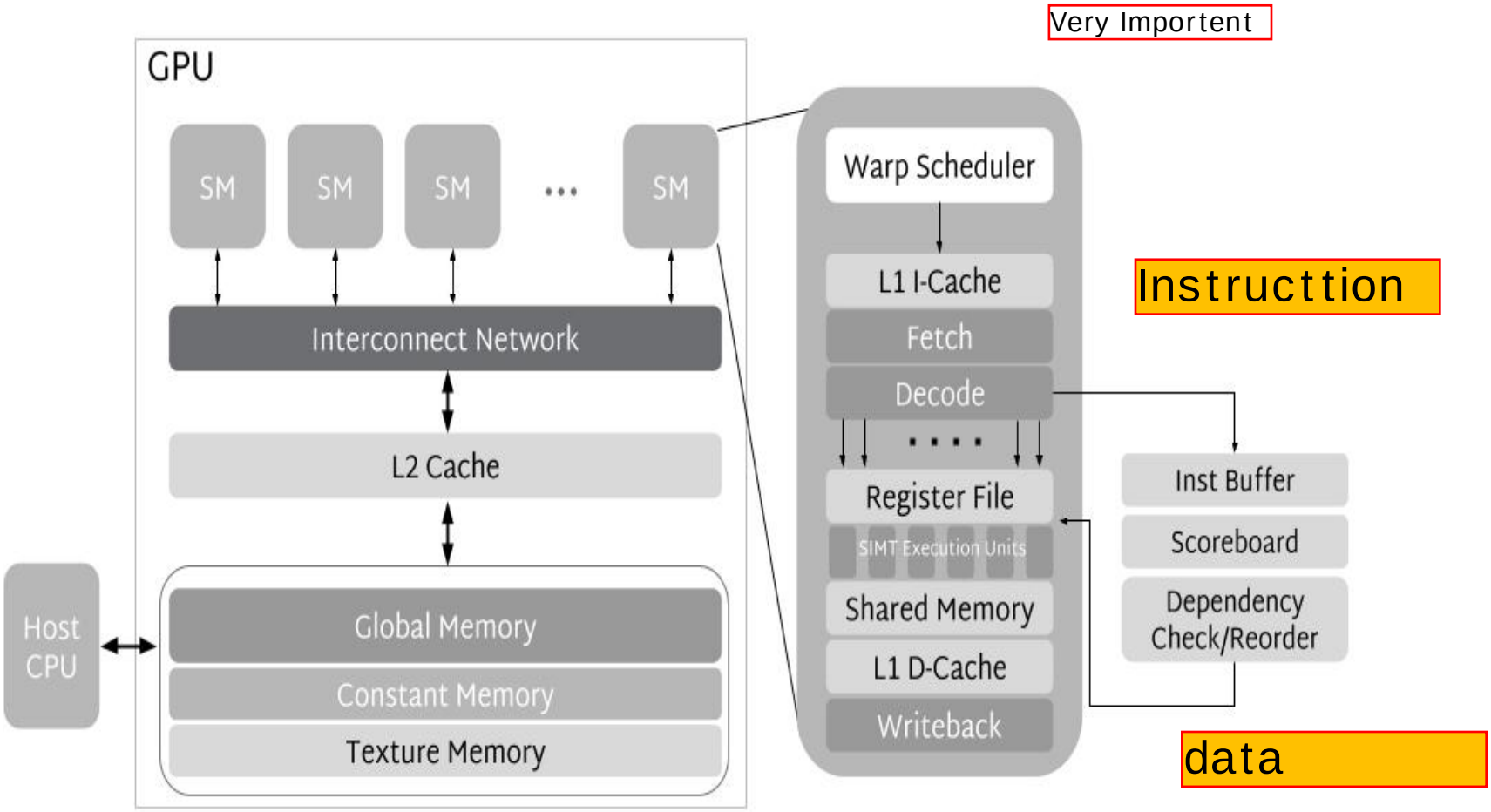
https://www.scalecomputing.com/resources/understanding-gpu-architecture

GPU and accelerator-based systems

Core Components

- Stream Processors – Parallel Processing Units
- Memory (VRAM) – Role in Data Storage and Fast Access
- Cache Hierarchy – L1, L2, and Shared Memory
- Memory Bus and Bandwidth – Impact on Performance
- Clock Speeds and Efficiency

GPU and accelerator-based systems



GPU vs CPU

Feature	GPU	CPU
Primary Function	Graphics rendering, parallel processing	General-purpose computing, sequential tasks
Architecture	Highly parallel architecture with thousands of cores	Fewer, more powerful cores optimized for sequential processing.
Parallel Processing	Suited for parallel tasks due to numerous cores.	Primarily designed for sequential processing tasks
Instruction Set	Limited instruction set, optimized for specific tasks (eg, matrix operations)	Comprehensive instruction set for a wide range of tasks.
Memory Hierarchy	Multiple levels of memory (global, shared, registers) for efficient parallel processing.	Typically has a cache hierarchy (L1, L2, L3 caches) optimized for sequential tasks.
Clock Speed	Lower base clock speeds but compensated by adequate number of cores.	Higher base clock speeds optimized for sequential tasks.
Performance per Core	Lower performance per core due to specialization in parallel tasks	Higher performance per core for sequential tasks
Flexibility	Specialized for graphics and parallel computation, less flexible for general computing.	Versatile and flexible for various computing tasks
Energy Efficiency	Less energy efficient due to the emphasis on parallel processing tasks	More energy efficient for sequential tasks
Usage Scenarios	Ideal for workloads that support parallel processing, like gaming, AI, scientific simulations, etc.	Suited for general computing tasks such as running operating systems, office applications, etc.
Cost	Relatively costlier, due to specialized hardware	Generally, more cost effective for general purpose computing
Programming Model	Requires specialized programming (e.g., CUDA, OpenCL) for optimal utilization.	Relatively standard programming languages (e.g., C, C++, Java) can be used.